arXiv:2110.11024v2 [cs.LG] 23 Nov 2021

Watermarking Graph Neural Networks based on Backdoor Attacks

Jing Xu
Delft University of Technology

Stjepan Picek
Delft University of Technology

Abstract

Graph Neural Networks (GNNs) have achieved promising performance in various real-world applications. Building a powerful GNN model is not a trivial task, as it requires a large amount of training data, powerful computing resources, and human expertise on fine-tuning the model. What is more, with the development of adversarial attacks, e.g., model stealing attacks, GNNs raise challenges to model authentication. To avoid copyright infringement on GNNs, it is necessary to verify the ownership of the GNN models.

In this paper, we present a watermarking framework for GNNs for both graph and node classification tasks. We 1) design two strategies to generate watermarked data for the graph classification and one for the node classification task, 2) embed the watermark into the host model through training to obtain the watermarked GNN model, and 3) verify the ownership of the suspicious model in a black-box setting. The experiments show that our framework can verify the ownership of GNN models with a very high probability (around 100%) for both tasks. In addition, we experimentally show that our watermarking approach is still effective even when considering suspicious models obtained from different architectures than the owner's.

1 Introduction

Many real-world data can be modeled as graphs with common examples, including social networks, gene interactions, and transport networks. Similar to the great success of deep learning technologies on, e.g., image recognition [10, 12, 20], speech recognition [8, 1], and natural language processing [7] tasks, deep graph models such as graph neural networks (GNNs) [11, 24, 9] have also achieved promising performance in processing graph data. Such successful results can be attributed to their superior ability to incorporate information from neighboring nodes in the graph recursively [26]. Still, building and training a well-performed graph neural network is not a trivial task, as it usually requires a large amount of training data, effort in designing and fine-tuning the model, and powerful computing resources, making the trained model having a monetary value. For example, the cost of training a machine learning model can be more than one million USD [21].

As graph neural networks are more widely developed and used, their safety and trustworthiness also become of a bigger concern. For instance, the adversary can steal the model through a model stealing attack. Recent works have shown the high effectiveness of model stealing attacks on complex models even without knowledge of the victim's architecture or the training data distribution [22, 17, 18]. Such model copyright infringement may damage the intellectual property (IP) of model owners, and if the model is intended to be released for commercial purposes, it will lead to financial loss for the model owners. Therefore, it is crucial to verify the ownership of a GNN model to protect intellectual property and prevent the leakage of the deep graph models. Additionally, similar to deep neural networks, GNNs are also vulnerable to backdoor attacks [27, 36]. For instance, in a Bitcoin transaction ego network, where the nodes are the transactions, and the edge between two nodes indicates the flow of Bitcoin from one transaction to another, the attacker can attack the GNNs to classify an illegal transaction as a legal one [25].

In this paper, instead of considering backdoor attacks for offensive purposes, we propose to use them to protect the IP of the GNN models. More precisely, we use the backdoor triggers as digital watermarks to identify the ownership of a GNN model. Indeed, digital watermarking is typically used to identify ownership of the copyright of media signals, e.g., audio, video, and image data [13]. There are already some works discussing embedding watermarks into DNN models to protect the IP of these models. For instance, Uchida et al. [23] pre-

sented a framework to embed watermarks into the parameters of deep neural networks via the parameter regularizer during the training process, leading to its white-box constraints. To address the limitations of watermarking DNNs in white-box setting, Zhang et al. [33] extended the threat model to support black-box setting verification for DNN models. These works all focus on the image domain, not graph data, so the watermarking mechanism can only generate watermark in image form, which cannot be embedded into a graph. For graph data, to the best of our knowledge, there is only one work about watermarking GNNs. Zhao et al. presented a watermarking framework for GNNs by generating a random graph associated with features and labels as the watermark [37]. However, this work only studies the watermarking in GNNs for the node classification task while neglecting other relevant settings, e.g., the graph classification task. Furthermore, the authors generated the watermark graph that is different from the original graph. Then, if the owner's model is trained by transductive learning, it is not feasible to embed the watermark into the host model since the graph structure of the training graph has been changed. Similarly, if a suspicious model is trained by transductive learning, verifying the ownership of the model by querying a random graph is not practical as the GNN model trained by transductive learning can only predict nodes on the graph that has the same structure as the graph it has been trained on.

Since the general idea of watermarking a GNN model in a black-box setting is to train the model using some specific samples so that the model can memorize the watermark information and be verified by the prediction of these samples, we propose to inject the backdoor trigger into specific samples to watermark the GNN model. More precisely, we design a watermarking framework for GNNs for two widely-used settings: graph classification task and node classification task, which includes three phases:

- **Watermarked data generation**. We designed two strategies to generate watermarked data for the graph classification task and one for the node classification task.
- **Watermark embedding**. We train the host model with the watermarked data. The intuition here is to explore the memorization capabilities of GNNs to learn the trigger pattern of the watermarked data automatically.
- **Ownership verification**. Once embedding the watermark into the model is done, we can verify the ownership of remote suspicious models by sending watermarked data generated in the first phase. Only the models protected by the watermarks are assumed to output matched predictions. To address the limitations of [37], we specifically use the feature trigger as the watermark pattern by modifying the feature information of the graph instead of changing the structure of the graph.

We evaluate our watermarking framework with four benchmark datasets: two for the node classification task and two for the graph classification task. The results show that our watermarking framework can verify the ownership of suspicious models with high probability (over 95%), while the performance of the watermarked GNN on its original task can be maintained. Our main contributions can be summarized as follows:

- We propose a watermarking framework to verify the ownership of GNN models for not only the node classification task but also the graph classification task. It is the first watermarking framework for GNNs on the graph classification task.
- We propose two watermark generation mechanisms to generate watermarked data for the graph classification task. One strategy is based on classical backdoor attacks, and we propose another strategy based on embedding watermark into random graphs, which experimentally shows superior performance.
- For the node classification task, in the watermarked data generation phase, we only modify the feature information of the graph, so there is no constraint on the training method of the host and suspicious models, i.e., it is also feasible for a model trained by transductive learning.
- We evaluate the proposed watermarking framework with several benchmark datasets and popular GNN models. We also explore a more reasonable metric to evaluate the watermarking performance of our method compared to the state-of-the-art. Experimental results show that the proposed method achieves promising performance in IP protection of the models while having a negligible impact on the original task. We also evaluate the robustness of our method concerning different models and against model modifications stemming from fine-tuning.

2 Background

This section introduces the relevant concepts of graph neural networks, backdoor attacks in GNNs, and digital watermarking in neural networks.

2.1 Graph Neural Networks (GNNs)

GNNs take a graph G as an input (including its structure information and node features) and learn a representation vector (embedding) for each node $v \in G$, z_v, or the entire graph, z_G. Modern GNNs follow a neighborhood aggregation strategy, where one iteratively updates

the representation of a node by aggregating representations of its neighbors. After k iterations of aggregation, a node's representation captures both structure and feature information within its k-hop network neighborhood. Formally, the k-th layer of a GNN is (e.g., GCN [11], GraphSAGE [9], and GAT [24]):

$$Z^{(k)} = AGGREGATE(A, Z^{(k-1)}; \theta^{(k)}). \quad (1)$$

Here, $Z^{(k)}$ represents the node embeddings in the matrix form computed after the k-th iteration and the $AGGREGATE$ function depends on the adjacency matrix A, the trainable parameters $\theta^{(k)}$, and the previous node embeddings $Z^{(k-1)}$. Finally, $Z^{(0)}$ is initialized as G's node features.

For the node classification task, the node representation $Z^{(k)}$ of the final iteration is used for prediction, while for the graph classification task, the $READOUT$ function pools the node embeddings from the final iteration K:

$$z_G = READOUT(Z^{(K)}). \quad (2)$$

$READOUT$ can be a simple permutation invariant function such as summation or a more sophisticated graph-level pooling function [31, 35].

The goal of the node-level classification task is that, given a single graph with partial nodes being labeled and others remaining unlabeled, GNNs can learn a robust model that effectively identifies the class labels for the unlabeled nodes [11]. In a node-level classification task, there are two types of training settings - inductive and transductive. In an inductive setting, the unlabeled nodes are not seen during training, while in a transductive setting, the test nodes (but not their labels) are also observed during the training process. The node classification task is used in many security applications, e.g., anomaly detection in a bitcoin transaction network or terrorist detection in a social network [4]. Graph-level classification aims to predict the class label(s) for an entire graph [35]. One practical application of the graph-level classification task is to detect whether a molecular is mutagen or not. Therefore, it is relevant to consider both settings when considering real-world applications.

2.2 Backdoor Attacks in GNNs

Deep Neural Networks (DNNs) are vulnerable to backdoor attacks [15, 14]. Specifically, a backdoored neural network classifier produces attacker-desired behaviors when a trigger is injected into a test example. Several studies showed that GNNs are also vulnerable to backdoor attacks [36, 27, 28]. Similar to the idea of backdoor attack in deep neural networks, the backdoor attack in GNNs is implemented by poisoning the training data with trigger (a subgraph with/without features or a subset of node feature), then training the GNN model with the trigger embedded data, and finally, predicting the test example injected with a trigger as the pre-defined target label.

2.3 Digital Watermarking in Neural Networks

Digital watermarking is a technique that embeds certain watermarks in carrier multimedia data such as audio, video, or images to protect their copyright [13]. The information to be embedded in a signal is called a digital watermark. The signal where the watermark is to be embedded is called the host signal. A digital watermarking system is usually divided into two steps: embedding and verification. Figure 1 shows a typical digital watermarking life cycle. In the embedding step, an embedding algorithm E embeds the watermark into the host signal to generate the watermarked data S_w. After embedding, the watermarked data is transferred or modified (dashed frame, this part is optional). During the watermark verification step, a verification algorithm is applied to attempt to extract the watermark from the watermarked signal. If the extracted watermark is equal to or within the acceptable distance to the original watermark, we can confirm that the signal is the protected signal.

At first, the goal of digital watermarking was to protect the copyright of multimedia data by embedding watermarks into the multimedia data. More recently, with the development of deep neural networks, new watermarking methods were designed to protect the DNN models by embedding watermarks into DNN models [23, 5, 2]. The idea of watermarking neural networks is similar to the traditional digital watermarking in multimedia data. To implement digital watermarking in neural networks, we can assume the signal in Figure 1 to be the model we want to protect, and the embedding function E and verification function V to be the training and inference phase of the protected model, and the modification function M to be some model modifications, e.g., fine-tuning.

3 GNN Watermarking

In this section, we discuss how to apply the backdoor attack to watermark GNN models. More precisely, we propose a framework to generate watermarked data, embed watermark into GNNs, and verify the ownership of GNNs through extracting watermark from them. The framework's purpose is to protect the IP of the graph neural networks by verifying the ownership of suspicious GNNs with embedded watermark. The framework first generates watermarked data and trains the host GNNs with the watermarked data. Through training, the GNNs automatically learn and memorize the connection between watermark pattern and the target label. As a result,

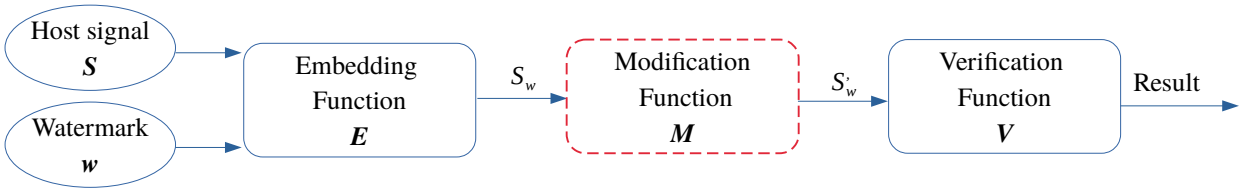


Figure 1: Digital watermarking life cycle.

only the model protected with our watermark can output pre-defined predictions, i.e., the assigned target label, when the watermark pattern is observed in the queries sent to the suspicious model.

Figure 2 illustrates the workflow of our GNN watermarking framework. The first step of the framework is to generate watermarked data. These watermarked data will be used for ownership verification later. Then the framework embeds generated watermark into the host GNN model m through training. Once trained with the watermark embedded data, the watermarked GNN model automatically learns the pattern of watermark and memorizes it. If such model is stolen, the owner of the model can easily verify it by querying watermark embedded samples as inputs and checking the outputs.

3.1 Threat Model

In our threat model, we model two parties, a model owner H, who owns a graph neural network model m for a certain task t, and a suspect S, who sets up a similar service t' from model m', where two services have a similar purpose $t \approx t'$. In practice, there are multiple ways for S to get the model m. For example, it could be an insider attack from the owner's organization H who leaks the model, or it could be stolen by various model stealing attacks, e.g., [22, 3]. The exact mechanism of how S obtains the model m is out of the scope of this paper.

In this paper, our goal is to help the model owner H to protect the intellectual property of model m. Intuitively, if model m is equivalent to m', we can confirm that S is a plagiarizer and m' is a plagiarized model of m [1]. In this paper, we define the owner's model m as the host model and the model m', which is likely to be stolen from m as the suspicious model. Note that m is not necessary the same as m'. Moreover, in this work, we assume that as an IP protector, we can only query the suspicious model m' in a black-box manner, i.e., we have no access to the network architecture and training data of the suspicious model m'. However, as the model owner H, we can have full access to the model m, including its architecture, training data, and the training process.

[1]The chances that various entities created the same model independently are small, especially when considering real-world, difficult tasks.

3.2 Watermarked data Generation

Graph classification. Since most graph classification tasks are implemented by utilizing GNNs to learn the network structure, we focus on utilizing the subgraph-based backdoor attacks [36, 27] to verify the ownership of GNNs for the graph classification task. Here, we investigate two watermarked data generation strategies for GNNs on the graph classification task.

1. **Embedding watermark into original training data**. Specifically, we select a subset of samples in training data and embed a generated random watermark (i.e., a random graph) into it. A random graph including n nodes and e edges is generally generated by starting with n isolated nodes and inserting e edges between nodes at random. A classical method to generate random graphs is called Erdos-Renyi (ER) random graph model [6] in which each possible edge occurs independently with a probability $p \in (0,1)$. This paper utilizes ER random graphs as the watermark graph for watermarking GNNs on the graph classification task.
   Once the watermark graph is generated, we embed it into each graph of the selected subset of training data by randomly choosing n nodes in the graph and changing their connections to be the same as the watermark graph. Since we only change the structure of the graph, we do not modify the node's features. The watermark information is also carried by the label of the watermark embedded graphs. We assume the value of the label of the watermarked graphs is in the range $[0, C)$, where C is the number of classes, and the label for watermarked graphs can be determined in advance. We emphasize that the labels of the sampled training data are different from the label of the watermark embedded graphs. In this way, three parameters - r (proportion of training data selected to be injected with the watermark graph), n (number of nodes in the watermark graph), and p (the probability of the edge existence in the watermark graph) would have a significant impact on the watermark generation, which will then affect

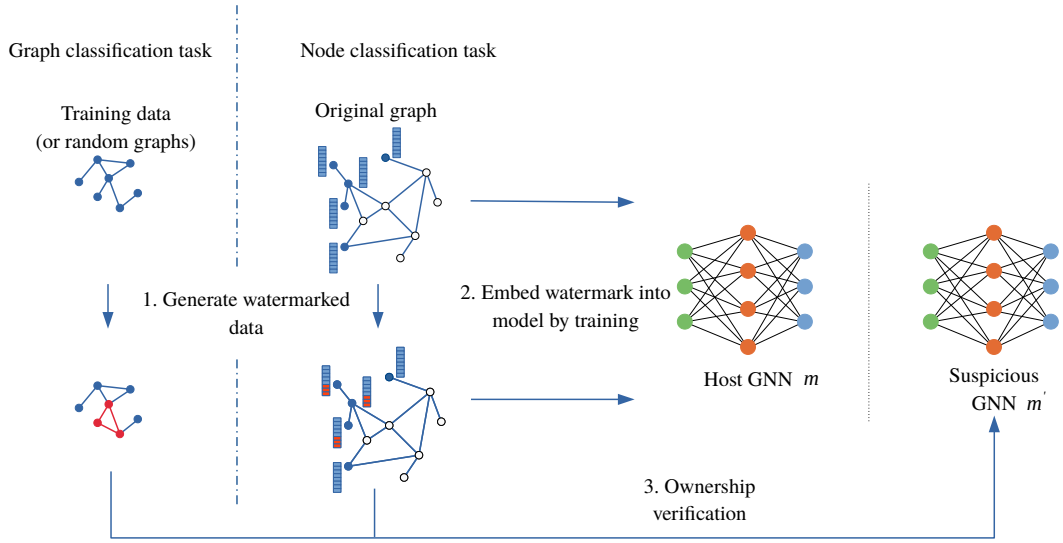


Figure 2: GNN watermarking framework.

the watermark embedding and verification later.

2. **Embedding watermark into generated random graphs**. In addition to embedding a watermark graph into the original training data, we propose first to generate random graphs and then embed generated watermark into these random graphs. The intuition here is that by embedding a watermark graph into the original training data, as discussed in the previous bullet, the watermark will have some side effects on the original functionality of our watermarked graph neural networks. To decrease the side effects on the original GNNs, we design this strategy. First, we generate a number of random graphs with the ER method, where we defined that number as a specific proportion (r) of the training data. The nodes number and edge existence probability are the same as the average node and edge number of the training data. Then, we generate the watermark graph in the same way described in the previous approach and embed the watermark graph into the random graphs generated in the first step. For the generated random graphs, we use the node degree as the node feature. We also assign the label for the watermark embedded graphs in advance, similar to the first strategy. Here, three parameters - r (proportion of training data to generate random graphs), n (number of nodes in the watermark graph), and p (the probability of the edge existence in the watermark graph) would have a significant impact on the watermark generation.

The detailed comparison and analysis of these parameters in the above two strategies are shown in Section 4.

Node classification. We investigate a watermarked data generation mechanism for GNNs on the node classification task based on the backdoor attack. The currently proposed backdoor attack on GNNs for the node classification task defines triggers as specific subgraphs, i.e., given an arbitrary subgraph g in G, by replacing g with the trigger (graph) g_t, the adversary attempts to force the unlabeled nodes within K hops to g to be misclassified into the target label y_t [27].

This backdoor attack is not practical if the host model is trained with transductive learning because once the graph structure is changed, the host model needs to train from scratch to learn the watermark pattern, which will be time- and source-consuming. Therefore, we apply the backdoor attack as proposed in [28] to implement watermarking GNNs for the node classification task. Specifically, we randomly select a proportion of the total number of nodes in the graph, denoted as r as the watermark carrier nodes, and change their subset node features into a pre-defined fixed value to generate the watermarked data. Given an arbitrary node in the graph, by changing the value of a subset of its features as a feature trigger and assigning a target label to it, the host model m aims to learn and memorize the watermark pattern.

3.3 Watermark Embedding

Graph classification. Once the watermarked data are generated, the next step in the framework is to embed the watermark into the host GNN model m. Here, we explore the intrinsic learning capability of graph neural networks to embed the watermark. The detailed GNN watermark embedding process is shown in Algorithm 1. The inputs are the original training data D_{train} and tar-

get label for the watermarked data, and the outputs are the watermarked GNN model m_w and watermarked data D_{wm}. The model owner defines the target label of the watermarked data. In the main function, we sample data D_{tmp} from the original training data. The data we sample has a label that is different from the target label (Line 3 in Algorithm 1). For the second watermarking strategy for the graph classification task, which is based on generating random graphs as watermark carrier data, we utilize the *ER* method to generate random graphs with an average number of nodes and edges of the training data and proportion r (Line 5 in Algorithm 1). Then, for each data in D_{tmp}, we add generated watermark to x and relabel it with y_t (Line 7-11 in Algorithm 1). Therefore, we obtain the watermarked data D_{wm}, which is used in the verification process later. We train the GNN model with both the original data D_{train} and D_{wm}. We assume that the GNN model will learn the watermark pattern during the training process and thus, be protected against model stealing attacks.

Algorithm 1: Watermark embedding for graph classification task

Input: Training set $D_{train} = \{x_i, y_i\}_{i=1}^{S}$, Target label $y_t \in [0, C)$
Output: Watermarked GNN model m_w, Watermark data D_{wm}

1 **Function** `WATERMARK_EMBEDDING()`:
2 $D_{wm} \leftarrow \emptyset$
3 $D_{tmp} \leftarrow sample(D_{train}, r, y \neq y_t)$
4 $(or$
5 $D_{tmp} \leftarrow GRAPH_GENERATE(n_{avg}, p_{avg}, r))$
6 **foreach** $d \in D_{tmp}$ **do**
7 $x_{wm} = ADD_WATERMARK(d[x], watermark)$
8 $y_{wm} = y_t$
9 $D_{wm} = D_{wm} \cup \{x_{wm}, y_{wm}\}$
10 **end**
11 **End Function**
12 $m_w = Train(D_{wm}, D_{train})$
13 **return** m_w, D_{wm}

Node classification. For the node classification task, the watermarking embedding process is similar to that in the graph classification task. The main difference is that now, we sample some nodes from the original training data and replace these nodes as the trigger nodes. Thus, instead of adding new training data to train the GNN model, as done for the graph classification task, we change a part of the training data and keep the size of the training data fixed.

3.4 Ownership Verification

After training our model with watermarked data, if adversaries steal and further fine-tune the watermarked model, they will likely set up an online service to provide the AI service of the stolen model. Therefore, it is difficult to access the architecture and parameters of the suspicious model directly. As we have analyzed in Section 3.1, in order to verify the ownership of the suspicious model m' in a black-box manner, we can send D_{wm}, which is returned in the previous watermark embedding process to the suspicious model. If most responses from m' equal to the Y_{wm} in D_{wm}, i.e., $m'(X_{wm}) \approx Y_{wm}$, we can assume that m' is from our watermarked model. The idea here is that the GNN model without embedding watermarks has no capacity to classify our watermark embedded data as the target label, so it is more likely for the model to classify the sent queries randomly. Therefore, the model from our watermarked model has a higher probability of classifying the sent queries correctly to the target label.

4 Experiments

In this section, we evaluate the performance of our watermarking mechanism in GNNs for graph classification and node classification tasks. The experiments are based on three datasets and two datasets for these two tasks, respectively. For each dataset, we train multiple models with and without watermark embedding.

4.1 Experimental Setting

Our experiments were run on a remote server with two NVIDIA 1080 Ti GPU processors having 32GB memory. We used the PyTorch framework for all the experiments, and each experiment was repeated ten times to obtain the average result.

Dataset. For the graph classification task, we use two publicly available real-world graph datasets, one chemical dataset and one social dataset: (i) NCI1 [16] - A subset of the dataset, consisting of chemical compounds screened for activity against non-small cell lung cancer. (ii) COLLAB [30] - a scientific collaboration dataset, derived from three public collaboration datasets.
For the node classification task, we also use two real-world datasets: (i) Cora [19] - a citation network in which each publication is described by a binary-valued word vector indicating the absence/presence of the corresponding word in the collection of $1,433$ unique words. (ii) CiteSeer [19] - another citation network with more nodes but less edges. Table 1 shows the statistics of these datasets.

Table 1: Datasets statistics.

Datasets	# Graphs	Avg. # nodes	Avg. # edges	Classes	Class Distribution
NCI1	4,110	29.87	32.30	2	2,053[0], 2,057[1]
COLLAB	5,000	74.49	2,457.78	3	2,600[0], 775[1], 1,625[2]
Cora	1	2,708	5,429	7	351[0], 217[1], 418[2], 818[3], 426[4], 298[5], 180[6]
CiteSeer	1	3,327	4,608	6	264[0], 590[1], 668[2], 701[3], 596[4], 508[5]

Dataset splits and parameter setting. For each graph classification dataset, we sample $2/3$ of the dataset as the training data and the rest as the test data. As we discuss in Section 3.2, the watermarked data is generated according to three parameters: watermarking rate r, watermark graph size n, and watermark graph density p. We set the watermark graph size n as the γ fraction of the graph dataset's average number of nodes. We then sample or generate an r fraction of the training data (with an untarget label) to embed the generated watermark. Since these parameters affect the watermarking performance in the graph classification task dramatically, we explore the impact of these parameters in Section 4.2.

For each node classification dataset, we use 20% of total nodes as the training data. We set the size of the feature watermark to be l and then sample r fraction of the training data to embed the generated feature watermark. The comparison of watermarking performance under different variants is shown in Section 4.2.

Models. In our experiments, we use three the state-of-the-art GNN models for graph classification task: DiffPool [31], GIN [29], and GraphSAGE [9]. For the node classification task, we use GCN [11] and GAT [24] as the host models. Since, as the model owner, we do not have access to the suspicious models, for each host model, we assume the suspicious model is one of the investigated models for that task, e.g., for the graph classification task, if we set the host model to be the DiffPool model, the suspicious could be DiffPool, GIN, and GraphSAGE. Due to page limitation, we show the results where the host model and the suspicious model are the same but later, we also conduct experiments of multiple host models and suspicious models to demonstrate the effectiveness of our GNN watermarking framework.

Metrics. The main purpose of our watermarking framework is to verify the ownership of the GNN model successfully. To achieve this goal, we first send queries of generated watermarked data to the watermarked model m_w and model m_{none} that is not watermarked, and then compare their performance on the watermarked data D_{wm}. If for δ of the queries in D_{wm}, $m_w(x_{wm}) = y_{wm}$ and $m_{none}(x_{wm}) \neq y_{wm}$, we can reach the conclusion that our watermarking framework can successfully verify the ownership. We emphasize that for the model without protection, there still exists a small probability of $m_{none}(x_{wm}) = y_{wm}$. Therefore, the larger the accuracy difference between the protected model m_p and model without protection m_{none} on the watermarked data, the higher probability we have to verify the ownership of the model successfully. We then set the accuracy difference as one of the metrics.

In addition to a good performance on the ownership verification task, a well-designed watermarking method should have only slight side effects on the host model's original task. We here also check whether our watermarking framework reduces the performance of the watermarked GNN model on its original task. Specifically, we compare the accuracy of watermarked models and clean models on the normal test data.

4.2 Results

Graph classification. As discussed in Section 3.2, there are two strategies for generating watermarked data D_{wm}. For each strategy, three parameters (watermarking rate r, watermark graph size n, and watermark graph density p) will affect the generated watermarked data and the final watermarking performance. The accuracy difference of different models on watermarked data with different variants(r, n, p) is shown in Figure 3. Here, $dataset$_t(r) means the model is trained with $dataset$ based on the first (second) watermarked data generation strategy, e.g., NCI1_t (NCI1_r) is the watermarked model trained by embedding watermark into sampled training data (generated random graphs). From Figures 3a and 3b, we can see that for the two datasets, with the increase of the watermarking rate and watermark graph size, the accuracy difference of all three models based on the first strategy is generally increasing, while that based on the second strategy keeps nearly 100%, except the results of the GIN model on the NCI1 dataset. It can be explained that with a higher watermarking rate and larger watermark size, there will be more training data embedded with the watermark so that the host model can learn the watermark pattern better. However, from Figure 3c, it can be observed that, in terms of edge existence probability in the

watermark graph from 0.4 to 1.0, the accuracy difference of DiffPool and GraphSAGE models on the NCI1 dataset almost keeps steady while that of the GIN model has a significant increase from $p = 0.8$ to $p = 1.0$. Next, in Figure 3c, we see that for the COLLAB dataset, the accuracy difference first decreases for the range $p = 0.4$ to $p = 0.8$ and then increases. This can be explained as when the watermark graph density is farther away from the graph density of the dataset, the trained model is more likely to recognize the watermark graph successfully. Therefore, the accuracy difference is the lowest when $p = 0.8$ for the COLLAB dataset, which has the density of 0.8978. Furthermore, there is no obvious difference for the NCI1 dataset, which has the density of 0.0755, except when $p = 1.0$. The reason may be that a fully connected watermark graph is "unique enough" that the host model can more easily learn its pattern.

Based on the analysis of the results in Figure 3 and also the later experimental results about the effect of watermarking GNNs on the original task, we set the parameters for the graph classification task as follows: $r = 0.15, \gamma = 0.2, p = 0.4$. Specifically, Table 2 shows the accuracy difference between model m_w and m_{none} on the watermarked data for the graph classification task with the selected parameters. D^t_{wm} is the watermarked data generated by embedding a watermark into sampled training data, while D^r_{wm} is the watermarked data generated by embedding a watermark into the generated random graphs. As we can see from Table 2, for datasets NCI1 and COLLAB, the accuracy difference of the GIN model is the lowest (80.18% and 56.90% respectively). One possible reason is that the GIN model has fewer redundant neurons than the other two models.

Table 2: Accuracy difference on watermarked data for graph classification task ($r = 0.15, \gamma = 0.2, p = 0.4$)

(a) NCI1

Accuracy Difference	DiffPool	GIN	GraphSAGE
D^t_{wm}	82.10%	80.18%	72.56%
D^r_{wm}	98.38%	84.93%	97.60%

(b) COLLAB

Accuracy Difference	DiffPool	GIN	GraphSAGE
D^t_{wm}	65.05%	56.90%	60.73%
D^r_{wm}	100.00%	100.00%	100.00%

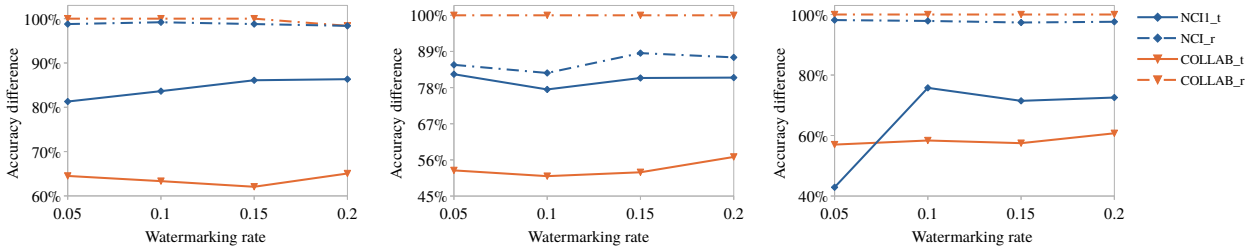
Node classification. For the node classification task, the generated watermarked data is decided by two parameters (watermarking rate r and feature watermark length l). The accuracy difference of two GNN models, i.e., GCN and GAT, on watermarked data with different variants is shown in Figure 4. As we can see from Figure 4a, for both datasets and models, the accuracy difference has a dramatic rise with r ranging from 0.05 to 0.15, and there is no obvious increase when r is in the range of $[0.15, 0.2]$. From Figure 4b, it can be observed that for both datasets, there is a dramatic increase between $l = 10$ and $l = 20$ and when l continue rising to $l = 50$, there is no effect for the CiteSeer dataset, and even for the Cora dataset, there is a small drop. It is reasonable since, with more nodes embedded with the feature watermark, the GNN model can learn the watermark pattern better, and also, with a larger watermark, the GNN model can better memorize the watermark pattern. Considering the results in Figures 4 and 7 (which will be further analyzed later), we set the parameters for the node classification task as follows: $r = 0.15, l = 20$. Table 3 shows the accuracy difference of GCN and GAT models on the watermarked nodes for the node classification task with selected parameters.

Table 3: Accuracy difference on watermarked data for node classification task ($r = 0.15, l = 20$)

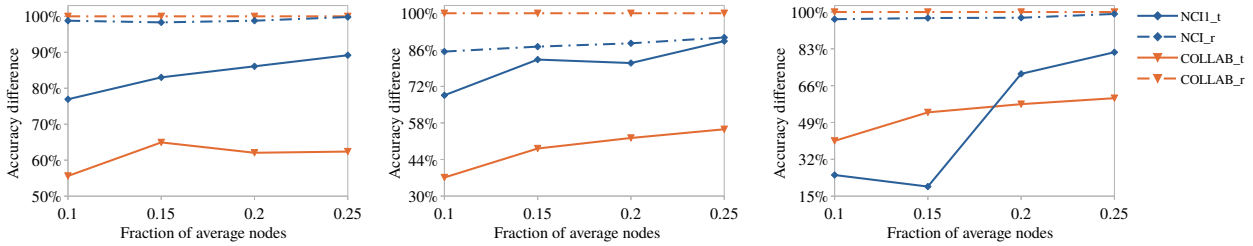
Accuracy Difference	GCN	GAT
Cora	97.70%	90.42%
CiteSeer	99.78%	99.25%

Here, we also compare our watermarking mechanism on the node classification task with the state-of-the-art [37]. Since in [37], the authors only use the watermark accuracy instead of the watermark accuracy difference to evaluate the watermarking performance, we also use this metric to compare the methods. The best watermark accuracy in that work is around 100%, and for most of our experiments, the watermark accuracy can also achieve 100%, as shown in Figure 5. However, it is important to emphasize that the intuition of setting accuracy difference on watermarked data instead of watermark accuracy as one of the evaluation metrics in this paper is that the non-watermarked model has the probability of also predicting the watermarked data as the predefined target label. In such a case, a high watermark accuracy cannot guarantee that the suspicious model is stolen from the host model, as shown from the difference between the left figure in Figure 4b and 5b. For the GCN model on the CiteSeer dataset, with the increase of the feature watermark length, the watermark accuracy keeps as high as 100%, but the accuracy difference is 88.53% at first and then increases to 100%. Therefore, it is more reasonable to use the accuracy difference of models on watermarked data to verify the performance of the watermarking framework.

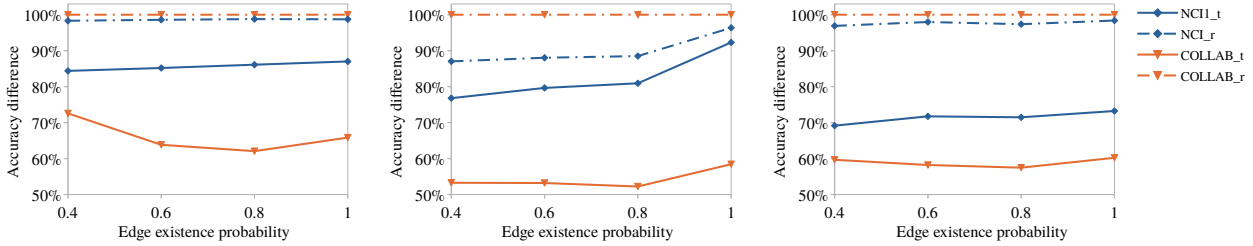
Effect on the original task. In order to measure the effect of our watermarking mechanism on the wa-



(a) with different watermarking rate r ($\gamma = 0.2, p = 0.4$)

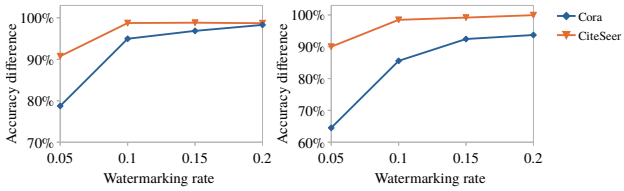


(b) with different fraction of average nodes γ ($r = 0.15, p = 0.4$)

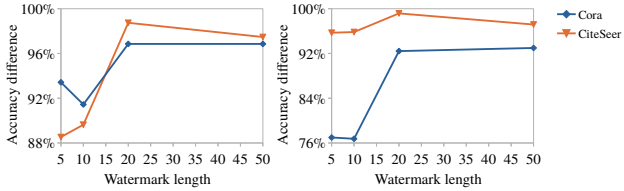


(c) with different edge existence probability p ($r = 0.15, \gamma = 0.2$)

Figure 3: Accuracy difference of DiffPool (left), GIN (center), GraphSAGE (right) models on watermarked data with different variants.

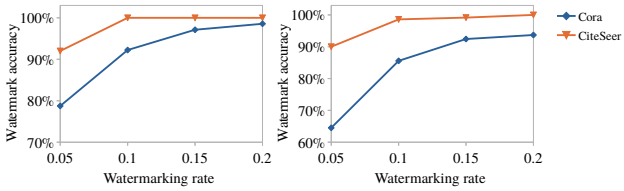


(a) with different watermarking rate r ($l = 20$)

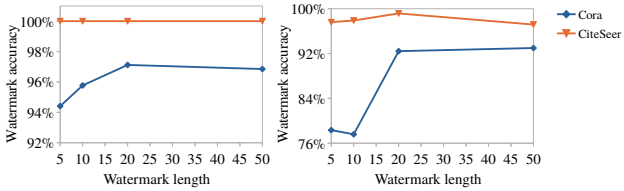


(b) with different watermark length l ($r = 0.15$)

Figure 4: Accuracy difference of GCN (left) and GAT (right) models on watermarked data with different variants.



(a) with different watermarking rate r ($l = 20$)



(b) with different watermark length l ($r = 0.15$)

Figure 5: Watermark accuracy of GCN (left) and GAT (right) models with different variants.

termarked model's original task, we measure the accuracy of models on the normal test data. The test data is not used to train the host model. Figures 6 and 7 illus-

trate the test accuracy of models with and without embedding watermarks under different variants for graph and node classification tasks, respectively. In Figure 6, CM_*dataset* represents the test accuracy of the clean model, e.g., CM_NCI1 is the test accuracy of the clean model trained on NCI1. We use the test accuracy of the clean model as the baseline, i.e., if the test accuracy of the watermarked model is close to the baseline, we can confirm that our watermarking mechanism will not affect the watermarked model's original task. We can see from Figures 6a and 6b that generally, with the increase of watermarking rate and watermark size, the test accuracy is decreasing for all models, which is reasonable because, with more data embedded with watermark, the model needs more neurons to learn the watermark patter and its original neurons for the original task are more likely to be modified. Therefore, there is a trade-off between the accuracy difference on watermarked data and the test accuracy. It can also be observed from Figure 6 that for all three models and two datasets, the test accuracy of the second watermarked data generation strategy is always much more closer to the baseline than the first strategy. This can be explained since by generating random graphs as the watermark carrier data in the watermark embedding phase, the model will try to explore redundancy network capacity to learn the watermark pattern while not affecting the original task. As for the first strategy, since the watermarked data is generated by embedding watermark into sampled training data, it is possible to embed a watermark into the graph structure, which has a critical effect on the final prediction, so the watermarking process will probably affect the parameters in the networks that are used for the original task. The results show that the second strategy has less effect on the model's original task. Combining those observations and the accuracy difference in Figure 3, we set $r = 0.15, \gamma = 0.2, p = 0.4$ so that we can achieve a high accuracy difference and test accuracy for most models and datasets. Similar to the graph classification task results, we set the parameters for the node classification task to $r = 0.15, l = 20$. Tables 4 and 5 show the test accuracy with the selected parameters for graph and node classification tasks, respectively.

Robustness on the model difference. As discussed in Section 3.1, we have no access and knowledge about the suspicious model m'. Therefore, the suspicious model is likely to have a different architecture from the host model m, i.e., m and m' are two different GNN models. An ideal watermarking framework should still be effective when the suspicious model is different from the host model, i.e., the watermarking framework should be robust to the model architecture difference. Figures 8 and 9 show the accuracy difference of the host model m and different suspicious model m' on the watermarked data, for the NCI1 and COLLAB datasets, respectively. Here, T represents the host model trained with the watermarked data generated by embedding the watermark into sampled training data, while R represents the host model trained with the watermarked data generated by embedding the watermark into generated random graphs. It is obvious from Figures 8 and 9 that for both datasets, the watermarking mechanism based on the second watermarked data generation strategy still has promising performance, i.e., nearly 100% on the accuracy difference, which is significantly higher than that of the first strategy. Especially when the host model is GIN, the accuracy difference of the second strategy and the COLLAB dataset is twice as large as that of the first strategy. This again emphasizes the superior performance of watermarking mechanism based on embedding watermarks into random graphs.

Table 6 shows the accuracy difference of different host and suspicious models on the watermarked data for the node classification task. As we can see, the accuracy difference is still high (more than 90%) even if the suspicious model is different from the host model. Therefore, our watermarking framework is robust to the architecture difference between the host and suspicious models.

Table 4: Test accuracy for graph classification task ($r = 0.15, \gamma = 0.2, p = 0.4$)

(a) NCI1

Test Accuracy	DiffPool	GIN	GraphSAGE
Clean Model	95.54%	95.66%	84.58%
D_{wm}^{t}	95.47%	93.57%	84.46%
D_{wm}^{r}	95.21%	93.96%	84.88%

(b) COLLAB

Test Accuracy	DiffPool	GIN	GraphSAGE
Clean Model	94.40%	93.91%	93.59%
D_{wm}^{t}	92.09%	91.10%	93.40%
D_{wm}^{r}	94.29%	92.31%	93.86%

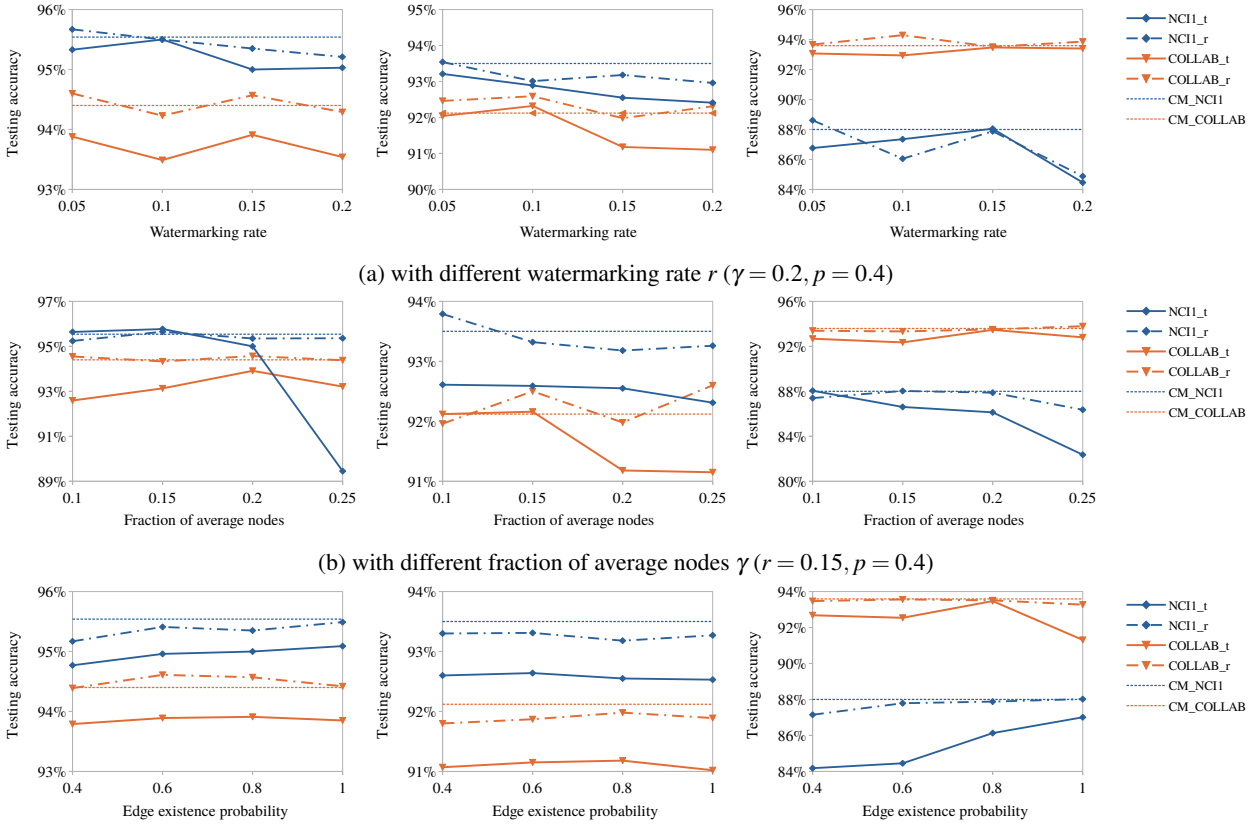
Table 5: Test accuracy for node classification task ($r = 0.15, l = 20$)

(a) Cora

Test Accuracy	GCN	GAT
Clean Model	83.67%	87.94%
D_{wm}^{t}	82.73%	86.98%

(b) CiteSeer

Test Accuracy	GCN	GAT
Clean Model	77.36%	80.66%
D_{wm}^{t}	76.98%	79.06%

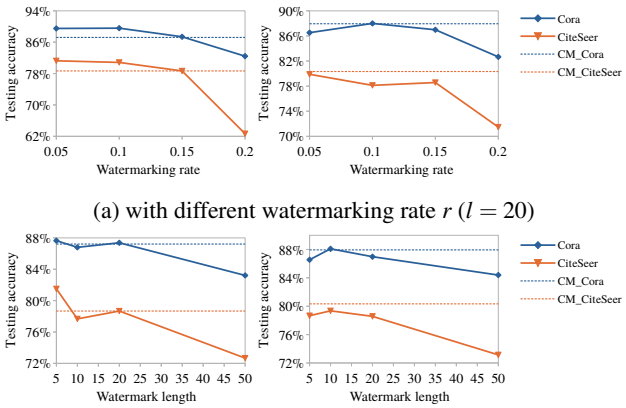


(a) with different watermarking rate r ($\gamma = 0.2, p = 0.4$)

(b) with different fraction of average nodes γ ($r = 0.15, p = 0.4$)

(c) with different edge existence probability p ($r = 0.15, \gamma = 0.2$)

Figure 6: Test accuracy of DiffPool (left), GIN (center), GraphSAGE (right) models with different variants.



(a) with different watermarking rate r ($l = 20$)

(b) with different watermark length l ($r = 0.15$)

Figure 7: Test accuracy of GCN (left), GAT (right) models with different variants.

Robustness against fine-tuning. As discussed before, training a well-performed GNN model from scratch requires a large amount of training data and powerful computing resources. Fine-tuning is a practical attack on GNNs since it can be used to apply existing state-of-the-art models to other but similar tasks with less effort than training a network from scratch when sufficient training data is not available [32]. Therefore, fine-tuning is likely to be used by a suspect to train a new model on top of the stolen model with only a small amount of training data.

Table 6: Accuracy difference of different suspicious models on watermarked data for node classification task

Accuracy Difference(%) *Cora*|*CiteSeer*		Suspicious model	
		GCN	GAT
Host model	GCN	97.90 99.78	97.76 100.0
	GAT	90.17 98.79	90.42 99.25

In this experiment, for each dataset, we use half of the test data to fine-tune the previously trained watermarked GNN model, and the second half of the test data is used to evaluate the new model. Then, we use the accuracy difference to determine whether the watermark embedded in the previously trained GNN model stayed effective in the new model. Also, the test accuracy is used to evaluate the performance of the newly trained model on its original task. Tables 7 and 8 show the accuracy dif-

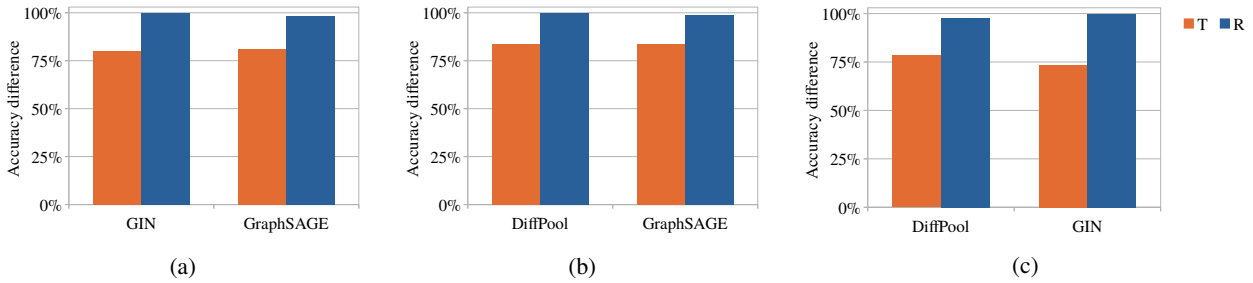


Figure 8: Accuracy difference of different suspicious models on two kinds of watermarked data for NCI1 dataset, with host model of DiffPool (a), GIN(b), GraphSAGE (c).

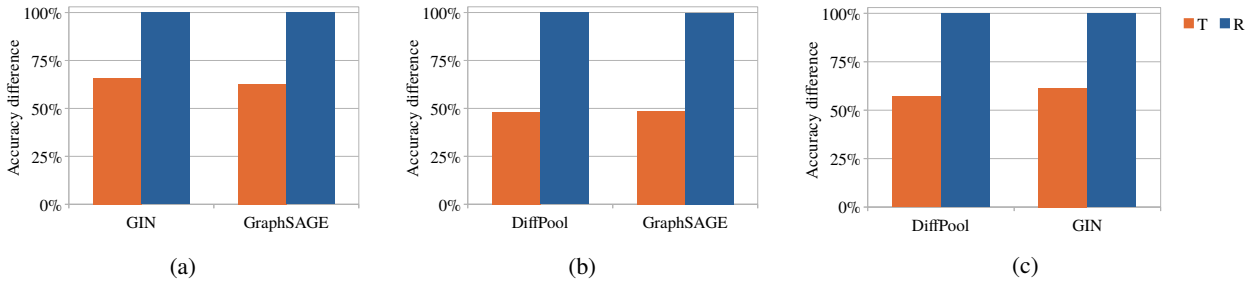


Figure 9: Accuracy difference of different suspicious models on two kinds of watermarked data for COLLAB dataset, with host model of DiffPool (a), GIN (b), GraphSAGE (c).

ference and test accuracy of the model after fine-tuning for graph classification task. Comparing the results from those two tables, we can see that fine-tuning does not reduce significantly (less than 3%) the accuracy difference on the watermarked data for both datasets and all models. Even for the COLLAB dataset, based on the second watermarked data generation strategy, the accuracy difference can still be high as 100%. As for the test accuracy, from Tables 8 and 4, we can observe that after fine-tuning the embedded watermark in the model still has only a slight effect on the model's original task. Tables 9 and 10 show the results for the node classification task, which are similar to that for the graph classification task.

Table 7: Accuracy difference on watermarked data after fine-tuning for the graph classification task

(a) NCI1

Accuracy Difference	DiffPool	GIN	GraphSAGE
D^t_{wm}	80.67%	79.31%	71.10%
D^r_{wm}	95.28%	82.56%	98.17%

(b) COLLAB

Accuracy Difference	DiffPool	GIN	GraphSAGE
D^t_{wm}	64.50%	55.22%	60.67%
D^r_{wm}	100.00%	100.00%	100.00%

5 Related Work

Watermarking neural networks. In addition to being used to hide the secret information into the multimedia signal to protect the copyright of those multimedia data, digital watermarking is also applied to protect the ownership of the neural networks. Recently, there are some works that studied the watermarking for deep neural networks [23, 2, 33, 34]. Uchida et al. used a parameter regularizer to embed a watermark in model parameters [23]. One limitation of this work is that it assumes the suspicious models can be accessible to extract all the parameters, which is not practical because the stolen models are usually used in a black-box setting, and it will be hard to get access to models' architecture and parameters directly. Adi et al. used random training instances and random labels to watermark a neural network in a black-box way [2]. This work is more related to ours since they also designed a backdoor attack to implement a watermarking neural network model. However, this work focuses on deep neural networks, which is different from the scenario we consider: verifying the ownership of graph neural networks. Zhang et al. investigated three DNN-applicable watermark generation algorithms [33]. Their work proposed a watermark implanting approach

Table 8: Test accuracy after fine-tuning for the graph classification task

(a) NCI1

Test Accuracy	DiffPool	GIN	GraphSAGE
Clean Model	95.54%	95.66%	84.58%
D_{wm}^{t}	93.65%	95.53%	82.76%
D_{wm}^{r}	93.15%	95.84%	82.89%

(b) COLLAB

Test Accuracy	DiffPool	GIN	GraphSAGE
Clean Model	94.40%	93.91%	93.59%
D_{wm}^{t}	92.10%	92.63%	93.03%
D_{wm}^{r}	92.88%	93.92%	93.31%

Table 9: Accuracy difference on watermarked data after fine-tuning for the node classification task

Accuracy Difference	GCN	GAT
Cora	95.28%	89.93%
CiteSeer	99.62%	99.04%

to infuse watermark into deep learning models, and they remotely verified the ownership of DNN models. Zhang et al. leveraged the spatial invisible watermarking mechanism to protect image processing models [34]. To enable watermarks from binary bits to high-resolution images, both traditional and deep spatial invisible watermarking mechanisms were used.

All the methods mentioned above focus on the watermarking technique for deep neural networks. Although recently, there is one work investigating watermarking graph neural networks [37], this research direction is still under-studied. Zhao et al. [37] presented a watermarking framework to GNNs by generating a random graph associated with features and labels as the trigger input. However, the work focuses on watermarking graph neural networks on the node classification task, while in our paper, we also design watermarking framework for GNNs on the graph classification task.

Backdoor attacks in GNNs. A number of recent papers have conducted backdoor attacks in GNNs. Zhang et al. proposed a subgraph-based backdoor attack to GNNs for the graph classification task [36]. Xi et al. presented a subgraph-based backdoor attack to GNNs, but this attack can be instantiated for both node classification and graph classification tasks [27]. Xu et al. [28] conducted the research on the explainability of the impact of trigger injecting position on the performance of backdoor attacks on GNNs, and they also proposed a new backdoor attack strategy for the node classification task. While backdoor attacks in GNNs are undesirable in most settings, we see here that they can be turned into a defense mechanism by applying it to the task of watermarking GNN models.

Table 10: Test accuracy after fine-tuning for the node classification task

Test Accuracy	GCN	GAT
Cora	82.95%	86.73%
CiteSeer	77.07%	79.96%

6 Conclusions and Future Work

This paper proposed a watermarking framework for graph neural networks, which includes generating watermarked data with different strategies, embedding the watermark into the host model through training, and verifying the ownership of the suspicious model by querying previously generated watermarked data. We designed watermarking mechanism for two GNN applications: the graph classification task and node classification task, and performed a comprehensive evaluation with our watermarking framework on different datasets and models. Through comprehensive experiments, we demonstrate that our method can achieve powerful watermarking performance while having a negligible effect on the host model's original task. We further demonstrate that our method is still effective when the suspicious model has different architectures from the host model and is robust against model modification: fine-tuning. For future work, we plan to evaluate our watermarking framework against more model modifications, e.g., model pruning, model compression. Moreover, we are also interested in exploring why in the graph classification task, the second watermarked data generation strategy has superior watermarking performance compared to the first strategy through some neural network explanation tools.

References

[1] Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Process. Mag.*, 29(6):82–97, 2012.

[2] Yossi Adi, Carsten Baum, Moustapha Cissé, Benny Pinkas, and Joseph Keshet. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In William Enck and Adrienne Porter Felt, editors, *27th USENIX Security Symposium, USENIX Security 2018, Baltimore, MD, USA, August 15-17, 2018*, pages 1615–1631. USENIX Association, 2018.

[3] Lejla Batina, Shivam Bhasin, Dirmanto Jap, and Stjepan Picek. CSI NN: reverse engineering of neural network architectures through electromagnetic side channel. In Nadia Heninger and Patrick Traynor, editors, *28th USENIX Security Symposium, USENIX Security 2019, Santa Clara, CA, USA, August 14-16, 2019*, pages 515–532. USENIX Association, 2019.

[4] Anshika Chaudhary, Himangi Mittal, and Anuja Arora. Anomaly detection using graph neural networks. In *2019 International Conference on Machine Learning, Big Data, Cloud and Parallel Computing (COMITCon)*, pages 346–350, 2019.

[5] Huili Chen, Bita Darvish Rouhani, Cheng Fu, Jishen Zhao, and Farinaz Koushanfar. Deepmarks: A secure fingerprinting framework for digital rights management of deep learning models. In Abdulmotaleb El-Saddik, Alberto Del Bimbo, Zhongfei Zhang, Alexander G. Hauptmann, K. Selçuk Candan, Marco Bertini, Lexing Xie, and Xiao-Yong Wei, editors, *Proceedings of the 2019 on International Conference on Multimedia Retrieval, ICMR 2019, Ottawa, ON, Canada, June 10-13, 2019*, pages 105–113. ACM, 2019.

[6] E. N. Gilbert. Random graphs. *The Annals of Mathematical Statistics*, 30(4):1141–1144, December 1959.

[7] Yoav Goldberg. A primer on neural network models for natural language processing. *J. Artif. Intell. Res.*, 57:345–420, 2016.

[8] Alex Graves, Abdel-rahman Mohamed, and Geoffrey E. Hinton. Speech recognition with deep recurrent neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2013, Vancouver, BC, Canada, May 26-31, 2013*, pages 6645–6649. IEEE, 2013.

[9] William L. Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 1024–1034, 2017.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*, pages 770–778. IEEE Computer Society, 2016.

[11] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[12] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In Peter L. Bartlett, Fernando C. N. Pereira, Christopher J. C. Burges, Léon Bottou, and Kilian Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25: 26th Annual Conference on Neural Information Processing Systems 2012. Proceedings of a meeting held December 3-6, 2012, Lake Tahoe, Nevada, United States*, pages 1106–1114, 2012.

[13] Gerrit C. Langelaar, Iwan Setyawan, and Reginald L. Lagendijk. Watermarking digital image and video data. A state-of-the-art overview. *IEEE Signal Process. Mag.*, 17(5):20–46, 2000.

[14] S. Li, M. Xue, B. Zhao, H. Zhu, and X. Zhang. Invisible backdoor attacks on deep neural networks via steganography and regularization. *IEEE Transactions on Dependable and Secure Computing*, pages 1–1, 2020.

[15] Yingqi Liu, Shiqing Ma, Yousra Aafer, Wen-Chuan Lee, Juan Zhai, Weihang Wang, and Xiangyu Zhang. Trojaning attack on neural networks. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018, San Diego, California, USA, February 18-21, 2018*. The Internet Society, 2018.

[16] Christopher Morris, Nils M. Kriege, Franka Bause, Kristian Kersting, Petra Mutzel, and Marion Neumann. Tudataset: A collection of benchmark datasets for learning with graphs. In *ICML 2020 Workshop on Graph Representation Learning and Beyond (GRL+ 2020)*, 2020.

[17] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. Knockoff nets: Stealing functionality of black-box models. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 4954–4963. Computer Vision Foundation / IEEE, 2019.

[18] Nicolas Papernot, Patrick D. McDaniel, Ian J. Goodfellow, Somesh Jha, Z. Berkay Celik, and Ananthram Swami. Practical black-box attacks against machine learning. In Ramesh Karri, Ozgur Sinanoglu, Ahmad-Reza Sadeghi, and Xun Yi, editors, *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security,*

AsiaCCS 2017, Abu Dhabi, United Arab Emirates, April 2-6, 2017, pages 506–519. ACM, 2017.

[19] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Gallagher, and Tina Eliassi-Rad. Collective classification in network data. *AI Mag.*, 29(3):93–106, 2008.

[20] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[21] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and Policy Considerations for Deep Learning in NLP. *CoRR*, abs/1906.02243, 2019.

[22] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. Stealing machine learning models via prediction apis. In *25th USENIX Security Symposium (USENIX Security 16)*, pages 601–618, Austin, TX, August 2016. USENIX Association.

[23] Yusuke Uchida, Yuki Nagai, Shigeyuki Sakazawa, and Shin'ichi Satoh. Embedding watermarks into deep neural networks. In Bogdan Ionescu, Nicu Sebe, Jiashi Feng, Martha A. Larson, Rainer Lienhart, and Cees Snoek, editors, *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval, ICMR 2017, Bucharest, Romania, June 6-9, 2017*, pages 269–277. ACM, 2017.

[24] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph Attention Networks. *International Conference on Learning Representations*, 2018. accepted as poster.

[25] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Robinson, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *CoRR*, abs/1908.02591, 2019.

[26] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, January 2021.

[27] Zhaohan Xi, Ren Pang, Shouling Ji, and Ting Wang. Graph backdoor. In Michael Bailey and Rachel Greenstadt, editors, *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, pages 1523–1540. USENIX Association, 2021.

[28] Jing Xu, Minhui Xue, and Stjepan Picek. Explainability-based backdoor attacks against graph neural networks. In Christina Pöpper and Mathy Vanhoef, editors, *WiseML@WiSec 2021: Proceedings of the 3rd ACM Workshop on Wireless Security and Machine Learning, Abu Dhabi, United Arab Emirates, July 2, 2021*, pages 31–36. ACM, 2021.

[29] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[30] Pinar Yanardag and S.V.N. Vishwanathan. Deep graph kernels. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '15, page 1365–1374, New York, NY, USA, 2015. Association for Computing Machinery.

[31] Zhitao Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L. Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 4805–4815, 2018.

[32] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In Zoubin Ghahramani, Max Welling, Corinna Cortes, Neil D. Lawrence, and Kilian Q. Weinberger, editors, *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada*, pages 3320–3328, 2014.

[33] Jialong Zhang, Zhongshu Gu, Jiyong Jang, Hui Wu, Marc Ph. Stoecklin, Heqing Huang, and Ian M. Molloy. Protecting intellectual property of deep neural networks with watermarking. In *Proceedings of the 2018 on Asia Conference on Computer and Communications Security, AsiaCCS 2018, Incheon, Republic of Korea, June 04-08, 2018*, pages 159–172. ACM, 2018.

[34] Jie Zhang, Dongdong Chen, Jing Liao, Han Fang, Weiming Zhang, Wenbo Zhou, Hao Cui, and Nenghai Yu. Model watermarking for image processing networks. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 12805–12812. AAAI Press, 2020.

[35] Muhan Zhang, Zhicheng Cui, Marion Neumann, and Yixin Chen. An end-to-end deep learning architecture for graph classification. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence, (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, pages 4438–4445. AAAI Press, 2018.

[36] Zaixi Zhang, Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong. Backdoor attacks to graph neural networks. In Jorge Lobo, Roberto Di Pietro, Omar Chowdhury, and Hongxin Hu, editors, *SACMAT '21: The 26th ACM Symposium on Access Control Models and Technologies, Virtual Event, Spain, June 16-18, 2021*, pages 15–26. ACM, 2021.

[37] Xiangyu Zhao, Hanzhou Wu, and Xinpeng Zhang. Watermarking graph neural networks by random graphs. In Asaf Varol, Murat Karabatak, and Ihan Varol, editors, *9th International Symposium on Digital Forensics and Security, ISDFS 2021, Elazig, Turkey, June 28-29, 2021*, pages 1–6. IEEE, 2021.